

Algoritmen en computationeel denken

Wat een algoritme is, hoe je er een ontwerpt en test, en de vier principes van computationeel denken.

1 Wat is een algoritme?

Een **algoritme** is een **stappenplan met instructies in een vaste volgorde**. Die volgorde ligt vast: je mag de stappen niet zomaar wisselen, en ontbreekt er één stap, dan **klopt het resultaat meestal niet meer**.

Een algoritme hoeft **niet** op een computer te draaien. Een **niet-digitaal** algoritme voer je zonder computer uit: een **recept** om een cake te bakken, een stappenplan om een boterham met choco te smeren. Een **digitaal** algoritme wel: de code van een computerspel, of een programma dat een lampje laat flikkeren.

Met een algoritme kan je van alles laten doen: controleren of een getal een **priemgetal** is, **getallen sorteren** van klein naar groot, een **rechthoek laten tekenen** op het scherm. Het werkt voor **eenvoudige én ingewikkelde** problemen.

2 Een algoritme ontwerpen



De stappen die de fiche noemt. **IPO** staat hier voor **input, process en output**: voor je een algoritme schrijft, bekijk je **wat erin gaat en wat eruit moet komen**. De **programmeertaal** waarin je het daarna vertaalt, is volgens de fiche **Scratch**.

Je **test** een algoritme uit **om te zien of het echt doet wat je wilde**. Geeft het niet het juiste resultaat, dan **stuur je het bij**. Testen is dus geen formaliteit achteraf maar een echte stap.

Een **flowchart** is een schema met pijlen dat de stappen van een algoritme toont. Het heet ook een **stroombiagram**, en het dient om de oplossing **visueel voor te stellen**.

vorm in een flowchart	wat ze betekent
een afgeronde rechthoek	het begin of het einde
een parallelogram	invoer of uitvoer
een rechthoek	een bewerking of een instructie
een ruit	een keuze: ja of nee
een pijl	de volgorde waarin je gaat

Volgens de vakfiche vertaal je je algoritme daarna in **Scratch**.

3 Computationeel denken

De vakfiche noemt **vier** principes van computationeel denken: **abstractie**, **decompositie**, **patroonherkenning** en **het algoritme** zelf. De eerste drie helpen je een **efficiënt** algoritme te schrijven.

principe	wat het is	voorbeeld
abstractie	details weglaten die er niet toe doen	leg je de weg naar school uit, dan laat je de kleur van de huizen weg
decompositie	een probleem in kleinere delen opsplitsen	bij een spel apart werken aan de beweging, de punten en het geluid
patroonherkenning	gelijkenissen tussen problemen zien	een spel met tien niveaus die sterk op elkaar lijken

Abstractie betekent dus juist **minder** details, niet meer. Decompositie maakt een groot probleem overzichtelijker, en patroonherkenning laat je een oplossing **hergebruiken** die je al kent.

Je gebruikt **decompositie** als een probleem **te groot is voor één keer**, als verschillende delen **los van elkaar** werken, en als je **met meerdere mensen samen** werkt.

4 Fouten zoeken

Een **bug** is een **fout in een programma**. Het opsporen en oplossen van zo'n fout heet **debuggen**.

Moet een programma een vierkant tekenen maar maakt het er maar drie zijden van, dan **debug** je het algoritme: je zoekt de stap die ontbreekt of fout staat.

Een fout vind je **niet** alleen door naar de code te kijken. Vaak zie je ze pas door het algoritme **uit te voeren** en te vergelijken met wat je verwachtte. Daarom is testen een eigen stap.

Efficiënt betekent **niet** dat een algoritme zoveel mogelijk stappen heeft, maar net zo weinig mogelijk. Een **korter** algoritme met dezelfde uitkomst is meestal beter, omdat er **minder fouten in kunnen sluipen**.

Onthoud dit

- Een algoritme is een stappenplan met instructies in een vaste volgorde.
- Niet-digitaal: een recept, een stappenplan. Digitaal: de code van een spel, een programma.
- Een flowchart of stroomdiagram stelt de oplossing visueel voor, met pijlen tussen de stappen.
- Ontwerpen: probleemstelling formuleren, analyseren met IPO, schrijven, testen en bijsturen.
- IPO = input, process, output: wat gaat erin en wat moet eruit komen?
- Je vertaalt je algoritme volgens de fiche in Scratch.
- Vier principes: abstractie, decompositie, patroonherkenning en het algoritme.
- Abstractie = details weglaten. Decompositie = opsplitsen. Patroonherkenning = gelijkenissen zien.
- Een bug is een fout in een programma; die opsporen en oplossen heet debuggen.
- Efficiënt is zo weinig mogelijk stappen: in een korter algoritme sluipen minder fouten.